Hibernate generates only one SQL update statement:

```
Query:  ["UPDATE post SET title = ? WHERE id = ?"],
Params: [('Post nr. 1', 1), ('Post nr. 2', 2), ('Post nr. 3', 3)]
```

Just like it was the case for batching insert statements, when updating entities of different types:

```
List<PostComment> comments = entityManager.createQuery(
    "select c " +
    "from PostComment c " +
    "join fetch c.post ", PostComment.class)
.getResultList();

comments.forEach(comment -> {
    comment.setReview(comment.getReview().replaceAll("Good", "Very good"));
    Post post = comment.getPost();
    post.setTitle(post.getTitle().replaceAll("no", "nr"));
});
```

Hibernate flushes the batched `PreparedStatement` before switching to an entity of a different type:

```
Query:  ["UPDATE post_comment SET post_id = ?, review = ? WHERE id = ?"],
Params: [(1, 'Very good', 2)]

Query:  ["UPDATE post SET title = ? WHERE id = ?"],
Params: [('Post nr. 0', 1)]

Query:  ["UPDATE post_comment SET post_id = ?, review = ? WHERE id = ?"],
Params: [(3, 'Very good', 4)]

Query:  ["UPDATE post SET title = ? WHERE id = ?"],
Params: [('Post nr. 1', 3)]

Query:  ["UPDATE post_comment SET post_id = ?, review = ? WHERE id = ?"],
Params: [(5, 'Very good', 6)]

Query:  ["UPDATE post SET title = ? WHERE id = ?"],
Params: [('Post nr. 2', 5)]
```

Analogous to ordering inserts, Hibernate offers the possibility of reordering batch updates as well:

```
<property name="hibernate.order_updates" value="true"/>
```

With this configuration in place, when rerunning the previous example, Hibernate generates only two update statements:

```
Query:  ["UPDATE post SET title = ? WHERE id = ?"],
Params: [('Post nr. 0', 1), ('Post nr. 1', 3), ('Post nr. 2', 5)]

Query:  ["UPDATE post_comment SET post_id = ?, review = ? WHERE id = ?"],
Params: [(1, 'Very good', 2), (3, 'Very good', 4), (5, 'Very good', 6)]
```

Batching versioned data

An entity is versioned if the `@Version` annotation is associated with a numerical or a timestamp attribute. The presence of the `@Version` attribute activates the implicit optimistic locking mechanism for update and delete statements. When the entity is updated or deleted, Hibernate includes the entity version in the where clause of the currently executing SQL statement. If the entity was modified by a concurrent transaction, the version of the underlying table row would not match the one supplied by the current running statement. The update count returned by an update or a delete statement reports the numbers of rows affected by the statement in question, and, if the count value is zero (or even less), a `StaleObjectStateException` is thrown.

Prior to Hibernate 5, JDBC batching was disabled for versioned entities during update and delete operations. This limitation was due to some JDBC drivers inability of correctly returning the update count of the affected table rows when enabling JDBC batching.

Validating the underlying JDBC driver support is fairly simple. Once the `hibernate.jdbc.batch_versioned_data` property is activated, if there is no optimistic locking exception being mistakenly thrown during a non-concurrent batch update, then the driver supports versioned JDBC batching.

Since Hibernate 5, the `hibernate.jdbc.batch_versioned_data` configuration property is enabled by default, and it is disabled when using a pre-12c Oracle dialect (e.g. Oracle 8i, Oracle 9i, Oracle 10g). Because the Oracle 12c JDBC driver manages to return the actual update count even when using batching, the `Oracle12cDialect` sets the `hibernate.jdbc.batch_versioned_data` property to `true`.

For Hibernate 3 and 4, the `hibernate.jdbc.batch_versioned_data` should be enabled if the JDBC driver supports this feature.

13.3 Batching delete statements

Considering that the `hibernate.jdbc.batch_size` configuration property is set, when running the following test case:

```
List<Post> posts = entityManager.createQuery(
    "select p " +
    "from Post p ", Post.class)
.getResultList();

posts.forEach(entityManager::remove);
```

Hibernate will generate a single `PreparedStatement`:

```
Query:  ["DELETE FROM post WHERE id = ?"],
Params: [(1), (2), (3)]
```

If the `Post` entity has a `@OneToMany PostComment` association, and since `CascadeType.REMOVE` is inherited from the `CascadeType.ALL` attribute, when the `Post` entity is removed, the associated `PostComment` child entities will be removed as well.

```
List<Post> posts = entityManager.createQuery(
    "select p " +
    "from Post p " +
    "join fetch p.comments ", Post.class)
.getResultList();

posts.forEach(entityManager::remove);
```

Even if the JDBC batching setting is enabled, Hibernate still issues each delete statement separately.

```
DELETE FROM post_comment WHERE id = 2
DELETE FROM post WHERE id = 1

DELETE FROM post_comment WHERE id = 4
DELETE FROM post WHERE id = 3

DELETE FROM post_comment WHERE id = 6
DELETE FROM post WHERE id = 5
```

> Once the HHH-10483[a] is resolved, Hibernate will support delete statement ordering.
>
> The `hibernate.jdbc.batch_versioned_data` property applies to batched deletes just like for update statements.
>
> [a] https://hibernate.atlassian.net/browse/HHH-10483

Fortunately, there are multiple workarounds to this issue. Instead of relying on `CascadeType.REMOVE`, the child entities can be manually removed before deleting the parent entities.

```
for (Post post : posts) {
    for (Iterator<PostComment> commentIterator = post.getComments().iterator();
            commentIterator.hasNext(); ) {
        PostComment comment = commentIterator.next();
        comment.setPost(null);
        commentIterator.remove();
    }
}
entityManager.flush();
posts.forEach(entityManager::remove);
```

Prior to deleting the `Post` entities, the Persistence Context is flushed to force the `PostComment` delete statements to be executed. This way, the Persistence Context does not interleave the SQL delete statements of the removing `Post` and `PostComment` entities.

```
Query:  ["DELETE FROM post_comment WHERE id = ?"],
Params: [(2), (4), (6)

Query:  ["DELETE FROM post WHERE id = ?"],
Params: [(1), (3), (5)]
```

A more efficient alternative is to execute a bulk HQL delete statement instead. First, the `PostComment` collection mapping must be modified to remove the `orphanRemoval`, as well as the `CascadeType.REMOVE` setting.

```
@OneToMany(cascade = {CascadeType.PERSIST, CascadeType.MERGE}, mappedBy = "post")
private List<PostComment> comments = new ArrayList<>();
```

Without removing the `orphanRemoval` and the `CascadeType.REMOVE` setting, Hibernate will issue a select statement for every child entity that gets removed. Not only the SQL statements are more effective (due to batching), but the flushing is also faster since the Persistence Context doesn't have to propagate the remove action.

With this new mapping in place, the remove operation can be constructed as follows:

```
List<Post> posts = entityManager.createQuery(
    "select p " +
    "from Post p ", Post.class)
.getResultList();

entityManager.createQuery(
    "delete " +
    "from PostComment c " +
    "where c.post in :posts")
.setParameter("posts", posts)
.executeUpdate();

posts.forEach(entityManager::remove);
```

This time, Hibernate generates only two statements. The child entities are deleted using a single bulk delete statement, while the parent entities are removed using a batched `PreparedStatement`.

```
Query:  ["DELETE FROM post_comment WHERE post_id in (? , ? , ?)"],
Params: [(1, 3, 5)]

Query:  ["DELETE FROM post WHERE id = ?"],
Params: [(1), (3), (5)]
```

The most efficient approach is to rely on database-level cascading. For this purpose, the `post_comment` table should be modified so that the `post_id` foreign key defines a `DELETE CASCADE` directive.

```
ALTER TABLE post_comment ADD CONSTRAINT fk_post_comment_post
FOREIGN KEY (post_id) REFERENCES post ON DELETE CASCADE
```

This way, the deletion operation can be reduced to simply removing the `Post` entities:

```
List<Post> posts = entityManager.createQuery(
    "select p " +
    "from Post p ", Post.class)
.getResultList();

posts.forEach(entityManager::remove);
```

Running the `Post` removal operation generates only one batched `PreparedStatement`:

```
Query:  ["DELETE FROM post WHERE id = ?"],
Params: [(1), (3), (5)]
```

Because the Hibernate `Session` is unaware of the table rows being deleted on the database side, it is good practice to avoid fetching the associations that will be removed by the database.

```
List<Post> posts = entityManager.createQuery(
    "select p " +
    "from Post p ", Post.class)
.getResultList();

List<PostComment> comments = entityManager.createQuery(
    "select c " +
    "from PostComment c " +
    "where c.post in :posts", PostComment.class)
.setParameter("posts", posts)
.getResultList();

posts.forEach(entityManager::remove);
comments.forEach(comment -> comment.setReview("Excellent"));
```

When running the test case above, Hibernate generates the following SQL statements:

```
Query:  ["UPDATE post_comment SET post_id=?, review=? WHERE id=?"],
Params: [(1, 'Excellent', 2), (3, 'Excellent', 4), (5, 'Excellent', 6)]

Query:  ["DELETE FROM post WHERE id=?"],
Params: [(1), (3), (5)]
```

Luckily, the `EntityDeleteAction` is the last action being executed during flushing, so, even if the `PostComment(s)` are changed, the update statement is executed before the parent deletion.

But if the Persistence Context is flushed before changing the `PostComment` entities:

```
List<Post> posts = entityManager.createQuery(
    "select p " +
    "from Post p ", Post.class)
.getResultList();

List<PostComment> comments = entityManager.createQuery(
    "select c " +
    "from PostComment c " +
    "where c.post in :posts", PostComment.class)
.setParameter("posts", posts)
.getResultList();

posts.forEach(entityManager::remove);

entityManager.flush();

comments.forEach(comment -> comment.setReview("Excellent"));
```

An `OptimisticLockException` will be thrown because the associated table rows cannot be found anymore.

```
Query:  ["DELETE FROM post WHERE id=?"],
Params: [(1), (3), (5)]

Query:  ["UPDATE post_comment SET post_id=?, review=? WHERE id=?"],
Params: [(1, 'Excellent', 2), (3, 'Excellent', 4), (5, 'Excellent', 6)]

o.h.e.j.b.i.BatchingBatch - HHH000315: Exception executing batch
[Batch update returned unexpected row count from update [0];
actual row count: 0; expected: 1]
```

Because the row count value is zero, Hibernate assumes that the records were modified by some other concurrent transaction and it throws the exception to notify the upper layers of the data consistency violation.

14. Fetching

While in SQL data is represented as tuples, the object-oriented Domain Model uses graphs of entities. Hibernate takes care of the object-relational impedance mismatch, allowing the data access operations to be expressed in the form of entity state transitions.

It is definitely much more convenient to operate on entity graphs and let Hibernate translate state modifications to SQL statements, but convenience has its price. All the automatically generated SQL statements need to be validated, not only for effectiveness but for ensuring their efficiency as well.

With JDBC, the application developer has full control over the underlying SQL statements, and the select clause dictates the amount of data being fetched. Because Hibernate hides the SQL statement generation, fetching efficiency is not as transparent as with JDBC. More, Hibernate makes it very easy to fetch an entire entity graph with a single query, and too-much-fetching is one of the most common JPA-related performance issues.

To make matters worse, many performance issues can go unnoticed during development because the testing data set might be too small, in comparison to the actual production data. For this purpose, this chapter goes through various Java Persistence fetching strategies, and it explains which ones are suitable for a high-performance data-driven application.

As explained in the JDBC ResultSet Fetching chapter, fetching too many rows or columns can greatly impact the data access layer performance, and Hibernate is no different.

Hibernate can fetch a given result set either as a projection or as a graph of entities. The former is similar to JDBC and allows transforming the `ResultSet` into a list of DTO (Data Transfer Objects). The latter is specific to ORM tools, and it leverages the automatic persistence mechanism.

Unfortunately too often, this distinction is being forgotten, and data projections are unnecessarily replaced by entity queries. There are multiple reasons why entity queries are not a universal solution for reading data:

1. If a graph of entities has been loaded, but only a subset of the whole graph is used during UI rendering, the unused data will become a waste of database resources, network bandwidth, and server resources (objects need to be created and then reclaimed by the Java Garbage Collector without serving any purpose).
2. Entity queries are more difficult to paginate especially if they contain child collections.
3. The automatic dirty checking and the optimistic locking mechanisms are only relevant when data is meant to be modified.

So, projections are ideal when rendering subsets of data (e.g. read-only tables, auto-scrolling selectors), while entity queries are useful when the user wants to edit the fetched entities (e.g. forms, im-place editing).

When loading a graph of entities, the application developer must pay attention to the amount of data being fetched and also the number of statements being executed.

As a rule of thumb, a transaction should fetch just as much data as required by the currently executing business logic. Fetching more data than necessary can increase response time and waste resources.

Fetching entity graphs is useful when the data access layer needs to modify the currently loading entities.

14.1 DTO projection

While Hibernate allows fetching projections as an `Object` array, it is much more convenient to materialize the `ResultSet` in a DTO projection. Unlike returning an `Object[]`, the DTO projection is type-safe.

Considering the following `PostCommentSummary` DTO type:

```
public class PostCommentSummary {

    private Number id;
    private String title;
    private String review;

    public PostCommentSummary(Number id, String title, String review) {
        this.id = id;
        this.title = title;
        this.review = review;
    }

    public PostCommentSummary() {}

    public Number getId() { return id; }

    public String getTitle() { return title; }

    public String getReview() { return review; }
}
```

When executing the following `PostCommentSummary` projection query:

```
List<PostCommentSummary> summaries = entityManager.createQuery(
    "select new " +
    "   com.vladmihalcea.book.hpjp.hibernate.fetching.PostCommentSummary( " +
    "       p.id, p.title, c.review ) " +
    "from PostComment c " +
    "join c.post p")
.getResultList();
```

Hibernate is only selecting the columns that are needed for building a `PostCommentSummary` instance.

```
SELECT p.id AS col_0_0_, p.title AS col_1_0_, c.review AS col_2_0_
FROM   post_comment c
INNER JOIN post p ON c.post_id = p.id
```

14.1.1 DTO projection pagination

Selecting too much data is a common cause of performance-related issues. A UI can display as much as the screen resolution allows it, and paginating data sets allows the UI to request only the info that is needed to be displayed in the current view. Pagination is also a safety measure since it sets an upper boundary on the amount of data that is fetched at once, and this is especially relevant when the tables being scanned tend to grow with time.

Pagination is a good choice even for batch processing because it limits the transaction size, therefore avoiding long-running transactions.

As explained in the JDBC ResultSet limit clause section, the SQL:2008 `ResultSet` pagination syntax hast started being supported since Oracle 12c, SQL Server 2012, and PostgreSQL 8.4, and many relational database systems still use a vendor-specific SQL syntax for offset pagination.

> In the SQL Performance Explained[a] book, Markus Winand explains why *keyset* pagination scales better than the default offset pagination mechanism. Unfortunately, Hibernate 5.1 does not support it, and the Keyset pagination requires executing a native SQL query instead.
>
> As long as the filtering criteria are highly-selective so that the scanning result set is relatively small, the offset pagination performs reasonably well.
>
> [a] http://sql-performance-explained.com/

For the offset pagination, JPA can insulate the data access layer from database-specific syntax quirks. First, the `ResultSet` size can be limited by calling `setMaxResults` which Hibernate

translates to a `Dialect`-specific statement syntax. While it would have been much easier for Hibernate to use the `setMaxRows` method of the underlying JDBC `Statement`, the database-specific query syntax is desirable since it can also influence the database execution plan.

When running the following projection query on a PostgreSQL database:

```
List<PostCommentSummary> summaries = entityManager.createQuery(
    "select new " +
    "   com.vladmihalcea.book.hpjp.hibernate.fetching.PostCommentSummary( " +
    "       p.id, p.title, c.review ) " +
    "from PostComment c " +
    "join c.post p " +
    "order by p.id")
.setFirstResult(pageStart)
.setMaxResults(pageSize)
.getResultList();
```

Hibernate generates the select statement as follows:

```
SELECT p.id AS col_0_0_, p.title AS col_1_0_, c.review AS col_2_0_
FROM   post_comment c
INNER JOIN post p ON c.post_id = p.id
ORDER BY p.id
LIMIT  10 OFFSET 20
```

In this particular example, the `LIMIT` and the `OFFSET` PostgreSQL directives are used to control the window of data that needs to be fetched by the currently executing query.

ORDER BY

Without the ORDER BY clause, the order of rows in a result set is not deterministic. However, in the pagination use case, the fetched record order need to be preserved whenever moving from one page to another. According to the SQL standard, only the ORDER BY clause can guarantee a deterministic result set order because records are sorted after being extracted.

In the context of pagination, the ORDER BY clause needs to be applied on a column or a set of columns that are guarded by a unique constraint.

14.1.2 Native query DTO projection

DTO projections can be fetched with native queries as well. When using JPA, to fetch a list of `PostCommentSummary` objects with an SQL query, a `@NamedNativeQuery` with a `@SqlResultSetMapping` is required:

```
@NamedNativeQuery(name = "PostCommentSummary",
    query =
        "SELECT p.id as id, p.title as title, c.review as review " +
        "FROM post_comment c " +
        "JOIN post p ON c.post_id = p.id " +
        "ORDER BY p.id",
    resultSetMapping = "PostCommentSummary"
)
@SqlResultSetMapping(name = "PostCommentSummary",
    classes = @ConstructorResult(
        targetClass = PostCommentSummary.class,
        columns = {
            @ColumnResult(name = "id"),
            @ColumnResult(name = "title"),
            @ColumnResult(name = "review")
        }
    )
)
```

To execute the above SQL query, the `createNamedQuery` method must be used:

```
List<PostCommentSummary> summaries = entityManager.createNamedQuery(
        "PostCommentSummary")
.setFirstResult(pageStart)
.setMaxResults(pageSize)
.getResultList();
```

Hibernate generating the following paginated SQL query:

```
SELECT p.id as id, p.title as title, c.review as review
FROM   post_comment c
JOIN post p ON c.post_id = p.id
ORDER BY p.id
LIMIT  10 OFFSET 20
```

While JPQL might be sufficient in many situations, there might be times when a native SQL query is the only reasonable alternative because, this way, the data access layer can take advantage of the underlying database querying capabilities.

A much simpler alternative is to use the Hibernate-native API which allows transforming the `ResultSet` to a DTO through Java Reflection:

```
List<PostCommentSummary> summaries = session.createSQLQuery(
    "SELECT p.id as id, p.title as title, c.review as review " +
    "FROM post_comment c " +
    "JOIN post p ON c.post_id = p.id " +
    "ORDER BY p.id")
.setFirstResult(pageStart)
.setMaxResults(pageSize)
.setResultTransformer(
    new AliasToBeanResultTransformer(PostCommentSummary.class))
.list();
```

Although JPA 2.1 supports Constructor Expressions for JPQL queries as previously illustrated, there is no such alternative for native SQL queries.

Fortunately, Hibernate has long been offering this feature through the `ResultTransformer` mechanism which not only provides a way to return DTO projections, but it allows to customize the result set transformation, like when needing to build an hierarchical DTO structure.

To fully grasp why sometimes native queries become a necessity, the next example uses a hierarchical model that needs to be ranked across the whole tree structure. For this reason, in the following example, a post comment score ranking system is going to be implemented.

The goal of such a system is to provide the user a way to view only the most relevant comments, therefore, allowing him to ignore comments that have a low score.

The post comment score system is going to use the following database table:

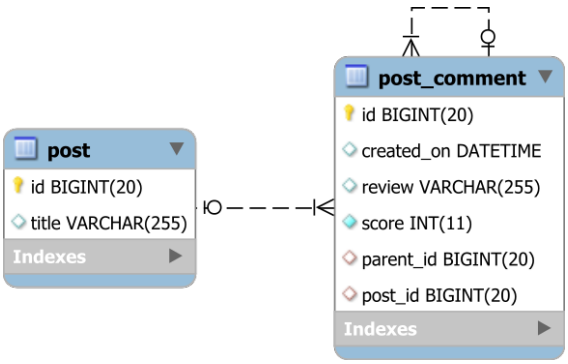

Figure 14.1:Post comment score ranking system tables

There is a one-to-many table relationship between `post` and `post_comment`. However, because users can also reply to comments, the `post_comment` table has also a one-to-one self-join table relationship.

The self-join association is commonly used for representing tree-like structures in a relational database. Additionally, each `post_comment` has a score which indicates its relevance.

The application in question needs to display the top-ranked comment hierarchies associated with a given post. The ranking can be calculated either in the data access layer or in the database, so it is worth comparing the performance impact of each of these two solutions.

The first approach uses application-level comment ranking, and, to minimize the fetching impact, a DTO projection is used to retrieve all records that need to be aggregated hierarchically.

```
List<PostCommentScore> postCommentScores = entityManager.createQuery(
    "select new " +
    "  com.vladmihalcea.book.hpjp.hibernate.query.recursive.PostCommentScore(" +
    "  pc.id, pc.parent.id, pc.review, pc.createdOn, pc.score ) " +
    "from PostComment pc " +
    "where pc.post.id = :postId ")
.setParameter("postId", postId)
.getResultList();
```

The associated SQL query looks as follows:

```
SELECT
    pc.id AS col_0_0_, pc.parent_id AS col_1_0_, pc.review AS col_2_0_,
    pc.created_on AS col_3_0_, pc.score AS col_4_0_
FROM  post_comment pc
WHERE pc.post_id = 1
```

The `PostCommentScore` DTO looks like this:

```
public class PostCommentScore {

    private Long id;
    private Long parentId;
    private String review;
    private Date createdOn;
    private long score;

    private List<PostCommentScore> children = new ArrayList<>();

    public PostCommentScore(Number id, Number parentId, String review,
        Date createdOn, Number score) {
        this.id = id.longValue();
        this.parentId = parentId != null ? parentId.longValue() : null;
        this.review = review;
        this.createdOn = createdOn;
        this.score = score.longValue();
    }

    public PostCommentScore() {}

    //Getters and setters omitted for brevity

    public long getTotalScore() {
        long total = getScore();
        for(PostCommentScore child : children) {
            total += child.getTotalScore();
        }
        return total;
    }

    public List<PostCommentScore> getChildren() {
        List<PostCommentScore> copy = new ArrayList<>(children);
        copy.sort(Comparator.comparing(PostCommentScore::getCreatedOn));
        return copy;
    }

    public void addChild(PostCommentScore child) {
        children.add(child);
    }
}
```

Once the `PostCommentScore` list is fetched from the database, the data access layer must extract the top-ranking comment hierarchies. For this, the sorting must be done in-memory.

```
1  List<PostCommentScore> roots = new ArrayList<>();
2
3  Map<Long, PostCommentScore> postCommentScoreMap = new HashMap<>();
4  for(PostCommentScore postCommentScore : postCommentScores) {
5      Long id = postCommentScore.getId();
6      if (!postCommentScoreMap.containsKey(id)) {
7          postCommentScoreMap.put(id, postCommentScore);
8      }
9  }
10 for(PostCommentScore postCommentScore : postCommentScores) {
11     Long parentId = postCommentScore.getParentId();
12     if(parentId == null) {
13         roots.add(postCommentScore);
14     } else {
15         PostCommentScore parent = postCommentScoreMap.get(parentId);
16         parent.addChild(postCommentScore);
17     }
18 }
19 roots.sort(
20     Comparator.comparing(PostCommentScore::getTotalScore).reversed()
21 );
22 if(roots.size() > rank) {
23     roots = roots.subList(0, rank);
24 }
```

The in-memory ranking process can be summarized as follows:

- Lines 4-10: Because the query does not use an `ORDER BY` clause, there is no ordering guarantee. Grouping `PostCommentScore` entries by their identifier must be done prior to reconstructing the hierarchy.
- Lines 12-20: The hierarchy is built out of the flat `PostCommentScore` list. The `PostCommentScore` map is used to locate each `PostCommentScore` parent entry.
- Lines 22-24: The `PostCommentScore` roots are sorted by their total score.
- Lines 26-28: Only the top-ranking entries are kept and handed to the business logic.

For many developers, this approach might be the first option to consider when implementing such a task. Unfortunately, this method does not scale for large `ResultSet(s)` because fetching too much data and sending it over the network is going to have a significant impact on application performance. If a `post` becomes very popular, the number of `post_comment` rows can easily skyrocket, and the system might start experiencing performance issues.

By moving the score ranking processing in the database, the `ResultSet` can be limited to a maximum size before being returned to the data access layer. Summing scores for all

comments belonging to the same `post_comment` root requires Recursive CTE queries and Window Functions, therefore, the following example uses PostgreSQL, and the database ranking logic looks like this:

```
List<PostCommentScore> postCommentScores = entityManager.createNativeQuery(
   "SELECT id, parent_id, root_id, review, created_on, score " +
   "FROM ( " +
   "   SELECT " +
   "      id, parent_id, root_id, review, created_on, score, " +
   "      dense_rank() OVER (ORDER BY total_score DESC) rank " +
   "   FROM ( " +
   "      SELECT " +
   "         id, parent_id, root_id, review, created_on, score, " +
   "         SUM(score) OVER (PARTITION BY root_id) total_score " +
   "      FROM (" +
   "        WITH RECURSIVE post_comment_score(id, root_id, post_id, " +
   "           parent_id, review, created_on, score) AS (" +
   "           SELECT " +
   "              id, id, post_id, parent_id, review, created_on, score" +
   "           FROM post_comment " +
   "           WHERE post_id = :postId AND parent_id IS NULL " +
   "           UNION ALL " +
   "           SELECT pc.id, pcs.root_id, pc.post_id, pc.parent_id, " +
   "              pc.review, pc.created_on, pc.score " +
   "           FROM post_comment pc " +
   "           INNER JOIN post_comment_score pcs ON pc.parent_id = pcs.id " +
   "        ) " +
   "        SELECT id, parent_id, root_id, review, created_on, score " +
   "        FROM post_comment_score " +
   "      ) score_by_comment " +
   "   ) score_total " +
   "   ORDER BY total_score DESC, id ASC " +
   ") total_score_group " +
   "WHERE rank <= :rank", "PostCommentScore")
.unwrap(SQLQuery.class)
.setParameter("postId", postId).setParameter("rank", rank)
.setResultTransformer(new PostCommentScoreResultTransformer())
.list();
```

As usual, a SQL query can be better understood if starting from the inner-most query:

- Lines 14-17: This query is the first one to be executed, and it selects the `post_comment` roots associated with the given `post` identifier.
- Line 18: The `UNION ALL` directive combines the previously generated result set with the current Recursive CTE projection.
- Lines 19-22: These lines represent the recursive step which, in this case, it joins the current `post_comment` rows with the previously scanned parents.
- Lines 12-13 and 24-25: The Recursive CTE is only a construct that needs to be explicitly called by a query. For this example, the `post_comment` hierarchy has a `root_id` which identifies all records belonging to the same comment root.
- Lines 8-11 and 26: This outer query is used to sum all scores for a given `post_comment` hierarchy. Unlike a regular `GROUP BY` clause, the Window Function allows aggregating the score without affecting the selected result set.
- Lines 4-7 and 27-28: This outer query is used to order the `post_comment` hierarchies by their overall score, and each hierarchy is given a top rank (e.g. 1, 2, 3).
- Lines 2-3 and 29-30: The outer-most query is only selecting the `post_comment` hierarchies that have a top rank higher than a given threshold.
- Line 31: The JPA `Query` is dereferenced to the underlying Hibernate-specific `SQLQuery` object.
- Line 33: Because the query was cast to an `SQLQuery` instance, the result can be transformed using the `ResultTransformer` utility.

Without using a `ResultTransformer`, Hibernate would return a `List` of `PostCommentScore` objects that need to be manually transformed into a tree structure, exactly like it was the case with the first DTO projection that was fetching all `PostCommentScore` records.

In the Recursive CTE use case, the result set is already ordered by the database so that the hierarchical structure can be constructed in a single iteration. This can also be done for the first example, but adding an `ORDER BY` directive is going to slow down the query execution significantly. When `ORDER BY` was added, the SQL query was 20 times slower even if the ordering was done by the entity identifier which was indexed by default.

For this reason, the first example did not feature a SQL `ORDER BY` clause, and the result set was, therefore, iterated twice. Compared to a SQL query, in-memory processing is blazing fast. For instance, processing around 35 000 `PostCommentScore` records takes around 2.5 milliseconds. On the other hand, fetching just 100 `PostCommentScore(s)` takes more than 3 milliseconds.

> In PostgreSQL, CTE is treated as an optimization fence[a], so caution is advised.
>
> [a] http://blog.2ndquadrant.com/postgresql-ctes-are-optimization-fences/

The `PostCommentScoreResultTransformer` looks as follows:

```
public class PostCommentScoreResultTransformer implements ResultTransformer {

    private Map<Long, PostCommentScore> postCommentScoreMap = new HashMap<>();

    private List<PostCommentScore> roots = new ArrayList<>();

    @Override
    public Object transformTuple(Object[] tuple, String[] aliases) {
        PostCommentScore commentScore = (PostCommentScore) tuple[0];
        Long parentId = commentScore.getParentId();
        if (parentId == null) {
            roots.add(commentScore);
        } else {
            PostCommentScore parent = postCommentScoreMap.get(parentId);
            if (parent != null) {
                parent.addChild(commentScore);
            }
        }
        postCommentScoreMap.putIfAbsent(commentScore.getId(), commentScore);
        return commentScore;
    }

    @Override
    public List transformList(List collection) {
        return roots;
    }
}
```

Having two options for the same data access logic requires a test to prove which one performs better. Considering that `n` is the number of root-level `post_comment` records, the following test creates comments on three levels, each upper level having twice as much entries as the immediate lower level, and the total number of `post_comment` entries is given by the following formula:

$$N = n + n \times \frac{n}{2} + n \times \frac{n}{2} \times \frac{n}{4}$$

To understand how each of these two options scales, the number of root-level `post_comment` entries varies from 4 to 8, 16, 24, 32, 48, and 64 records. By applying the mathematical formula above, the total number of `post_comment` records contained within one hierarchy can vary from 20 to 104, 656, 2040, 4640, 15024, and 34880 rows. Increasing the `ResultSet` size, the impact of fetching too much data becomes more and more apparent. On the other hand, even if it still needs to scan a lot of records, the database-level processing can avoid the fetching penalty.

The following graph captures the results when running these two score ranking data processing alternatives:

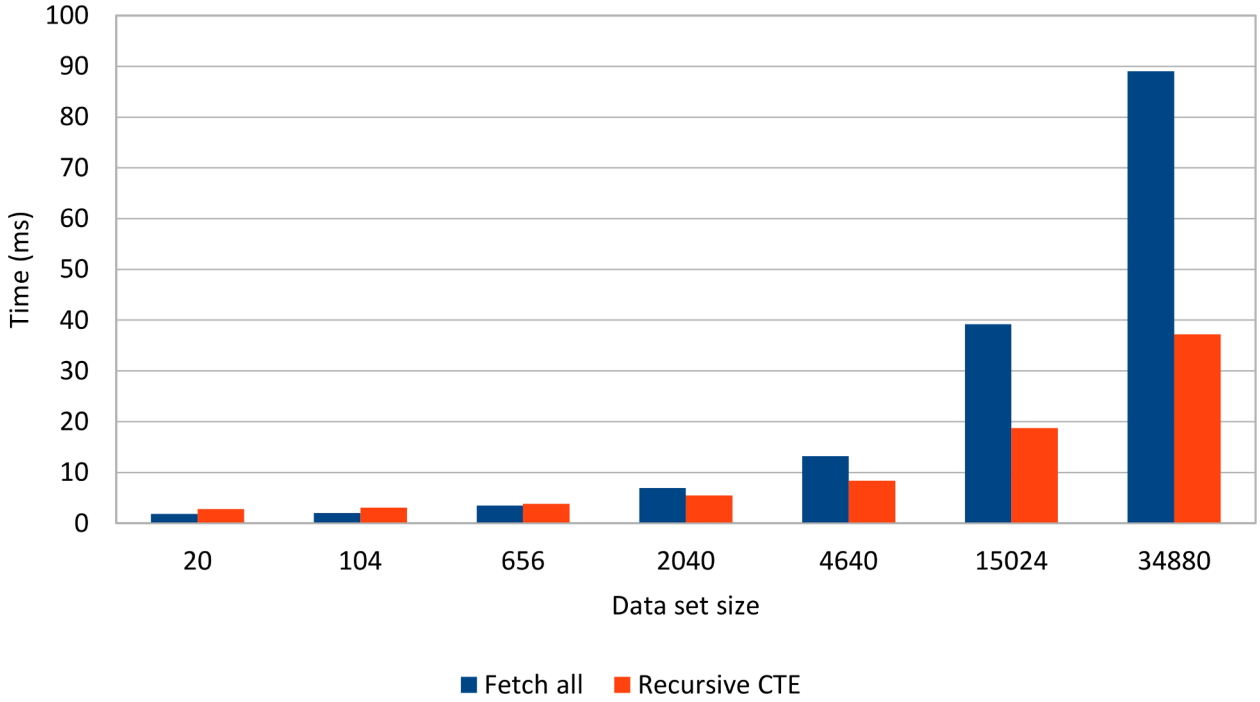

Figure 14.2: Fetching all records vs Recursive CTE

If the number of `post_comment` entries is low, the application-level processing will perform very well, even better than the Recursive CTE query. However, the larger the `ResultSet`, the more advantageous the database-processing alternative becomes. This graph is a good reminder that moving processing logic closer to the data set is a performance optimization that is worth considering.

Stored procedures

While SQL queries are ideal for fetching data projections, stored procedures and database functions can be very useful for processing data. Some complex database processing tasks can only be expressed through a procedure language (e.g. PL/SQL, T-SQL, PL/pgSQL) if the task requires mixing loops, conditional statements, arrays, or temporary tables.

Unlike SQL queries which are executed in the scope of the currently running transaction, stored procedure can also manipulate transaction boundaries. This can be very handy when trying to break an otherwise long-running transaction into smaller batches which can better fit the undo log memory buffers.